3DHeatTransfer: A High-Performance Multi-Backend Solver for 3D Heat Conduction with STL Geometry Support

20 April 2026

Summary

`HeatTransfer3D` is a lightweight, high-performance C++ solver for 3D heat conduction problems. It imports arbitrary 3D domains from STL files (with patch assignment: inlet, outlet, and wall) and supports mixed Dirichlet and Neumann boundary conditions.

The software provides **four interchangeable solver backends** — CPU and CUDA implementations of both stencil-based and matrix-based Jacobi iterative solvers — allowing users to select the best combination of speed, memory usage, and hardware availability. Results are exported in VTK format for easy visualization in ParaView.

The software targets researchers and engineers working in thermal management, energy systems, materials processing, electronics cooling, and related fields.

Statement of Need

Modeling three-dimensional heat conduction in complex geometries is a common requirement in applications such as electronics cooling, battery thermal management, additive manufacturing, and heat exchanger design. Despite its importance, existing tools present a trade-off between accessibility and flexibility.

Commercial multiphysics platforms such as ANSYS and COMSOL Multiphysics provide comprehensive capabilities but are often inaccessible due to licensing costs and complexity. Open-source frameworks such as OpenFOAM (Weller et al. 2007; Jasak, Jemcov, and Tukovic 2007) and FEniCS (Logg, Mardal, and Wells 2012) are highly flexible, but they are designed for general-purpose multiphysics simulations and require substantial setup even for pure conduction problems.

In contrast, many lightweight finite-difference solvers are restricted to regular

Cartesian domains and lack support for complex geometries, mixed boundary conditions, or hardware acceleration. GPU-enabled implementations do exist, but they are often research prototypes without extensible design or ease of use.

`HeatTransfer3D` addresses these limitations by providing a **lightweight, focused, and high-performance open-source solver** dedicated exclusively to steady-state and transient 3D heat conduction. Key innovations include: - Direct import of complex geometries from STL files with automatic boundary patch detection, - Support for mixed Dirichlet and Neumann boundary conditions, - Four interchangeable solver backends (CPU/GPU stencil-based and matrix-based Jacobi solvers) that let users balance memory usage, stability, and speed on different hardware, - Simple command-line interface and VTK output for seamless integration with ParaView.

This makes the software particularly suitable for rapid prototyping, parametric studies, and educational use, where ease of setup and computational efficiency are both critical.

State of the Field

Existing tools for heat conduction simulation can be broadly categorized into three groups: general-purpose multiphysics frameworks, mesh-based finite element tools, and specialized research codes.

General-purpose frameworks such as OpenFOAM (Weller et al. 2007; Jasak, Jemcov, and Tukovic 2007) and FEniCS (Logg, Mardal, and Wells 2012) provide extensive flexibility and support for coupled physics problems. However, they typically require substantial configuration effort, including mesh generation, solver selection, and case setup, which can be excessive for problems focused solely on heat conduction.

On the other end of the spectrum, lightweight finite-difference and finite-volume solvers are computationally efficient and widely used for heat conduction and diffusion-dominated problems due to their straightforward implementation on structured Cartesian or logically structured grids (LeVeque 2007; Patankar 1980). More advanced structured-grid frameworks, such as OpenSBLI (Howell and others 2016), extend this class of methods to high-performance computing environments. However, such frameworks are designed as general-purpose PDE toolchains and typically require additional abstraction layers and configuration to define application-specific workflows.

A number of GPU-accelerated finite-difference approaches have been developed to improve performance on structured grids. For example, phase-change heat conduction simulations on GPUs demonstrate substantial acceleration through optimized stencil execution, but do not support arbitrary or unstructured geometry representations (Liu et al. 2022). Similarly, fast and interactive GPU-based heat conduction simulators have been proposed for two-dimensional problems,

prioritizing real-time performance and interactivity over geometric generality and flexibility (Wei and others 2014). These approaches are typically designed around specific research applications and optimized stencil implementations, and do not provide modular solver architectures for extensible simulation workflows.

In addition, simplified and educational solvers such as FDiff3 emphasize numerical understanding and clarity of implementation for heat conduction problems, primarily in pedagogical settings (Russell and Probert 2004). Similarly, Python-based educational frameworks for two-dimensional heat transfer focus on accessibility and teaching purposes rather than modularity or extensibility (Kok et al. 2024).

Meshless approaches such as radial basis function finite differences (RBF-FD) enable simulation on scattered nodes and can handle arbitrary three-dimensional geometries without requiring structured grids (Fornberg and Flyer 2015; Miotti and others 2021). However, these methods are typically CPU-based, involve increased formulation complexity, and require careful parameter selection.

HeatTransfer3D occupies a middle ground between these categories by combining:

The geometric flexibility typically associated with mesh-based workflows (via STL-based preprocessing), The simplicity and efficiency of structured finite-difference discretization, A multi-backend design enabling both CPU and GPU execution, A lightweight, focused implementation tailored specifically for heat conduction.

This combination distinguishes it from both heavyweight multiphysics frameworks and minimal research or educational prototypes.

Software Design

The codebase follows a clean, modular structure written in modern C++17 with optional CUDA support. The main directories are organized as follows:

- **`src/`**: Core implementation files, including the main driver, geometry processing, boundary condition handling, solver kernels, and VTK output routines.
- **`include/`**: Header files defining classes and functions for the grid, solvers, STL importer, and utilities.
- **`tests/`**: Unit and integration tests.
- **`stlFiles/`**: Sample geometries (cube, cylinder, L_Channel, semiCylinder) with associated boundary patch files.

The solver uses a uniform Cartesian grid with finite-difference discretization of the heat equation. Two fundamental algorithmic approaches are implemented:

1. **Stencil-based solvers** — Direct Jacobi iterations on the temperature field (low memory footprint, high performance).

2. **Matrix-based solvers** — Explicit assembly of a sparse coefficient matrix followed by Jacobi iterations (more flexible for extensions).

Each approach has both a CPU and CUDA (GPU) implementation. Users can select any of the four backends at runtime using a simple command-line flag (`--solver 1..4`). CUDA kernels are optimized for coalesced memory access and make use of shared memory where beneficial. The geometry pipeline automatically detects and labels boundary patches from separate STL files, simplifying the application of different boundary conditions.

CMake is used for building, with convenient `build.sh` and `installDeps` scripts provided.

Performance

One of the key strengths of **HeatTransfer3D** is its **multi-backend design**, which allows users to dynamically select the most suitable solver based on the available hardware and problem size.

Performance comparisons for a representative test case (cube geometry with steady-state convergence) are shown below:

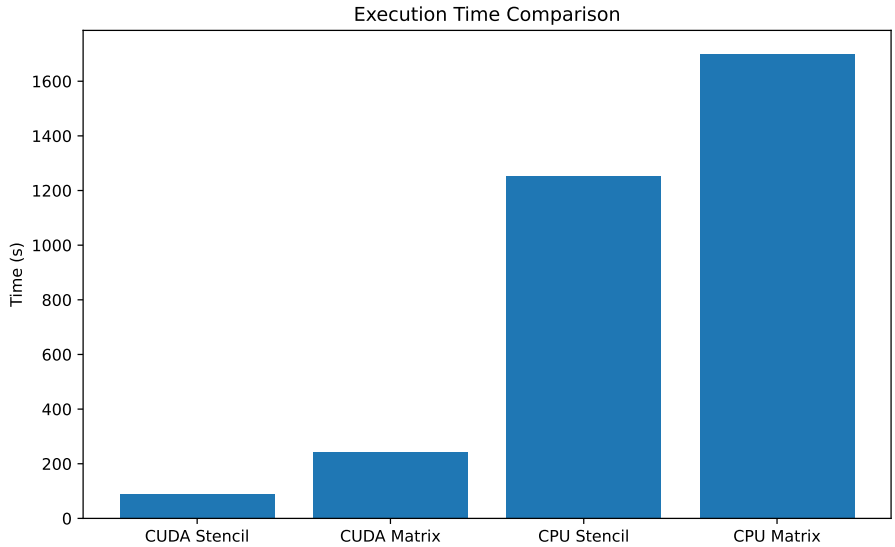

Figure 1: Execution time comparison of the four solvers for a $100 \times 100 \times 100$ grid (lower is better). CUDA backends demonstrate substantial speedups over CPU implementations.

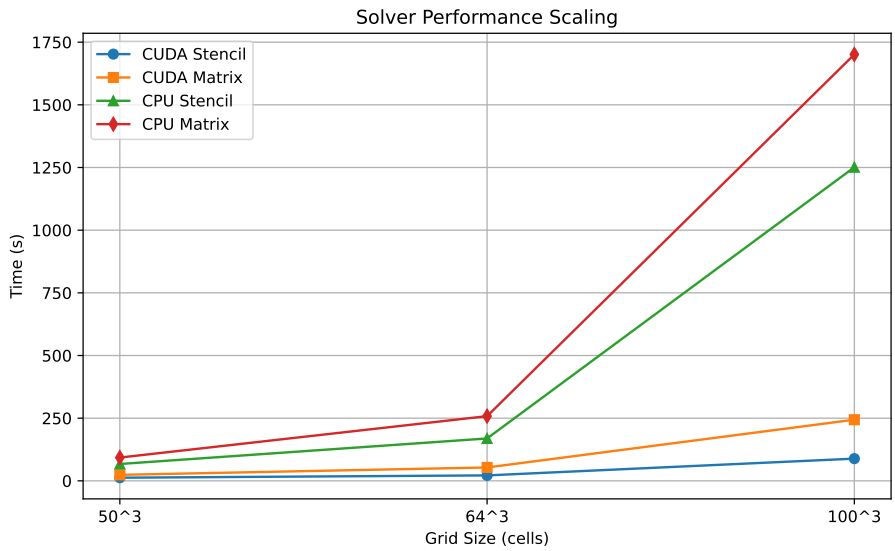

Figure 2: Strong scaling with increasing grid resolution (50^3 to 150^3). The GPU stencil backend maintains strong performance due to its low memory overhead.

All tests were conducted on the following hardware: CPU — 11th Gen Intel® Core™ i5-11400H @ 2.70 GHz; GPU — NVIDIA GeForce RTX 3050. The global tolerance was set as 1×10^{-6}. The CUDA stencil solver achieves an **8–20× speedup** compared to the CPU stencil solver, while still providing reliable performance on CPU backends. This flexibility makes the software suitable for both rapid prototyping and large-scale simulations.

Simulations

Here the few shapes simulated by solver will be displayed

Research Impact Statement

By offering high performance together with flexible solver selection, **HeatTransfer3D** enables faster design iterations and parametric studies in thermal engineering research. The open-source MIT license and modular structure also facilitate community contributions and integration into larger multiphysics workflows.

AI Usage Disclosure

AI-based tools were used to assist with debugging, syntax support, and language refinement during development and writing of this work. All scientific decisions,

implementation design, and results validation were performed and verified by the authors.

The following tools were used: - ChatGPT (GPT-4o) - Grok

Acknowledgements

We thank the open-source community for the tools that made this project possible.

References

Fornberg, Bengt, and Natasha Flyer. 2015. *Solving Pdes with Radial Basis Functions*. Cambridge University Press.

Howell, Jonathan, and others. 2016. "OpenSBLI: An Open-Source Framework for the Automated Derivation and Parallel Execution of Finite Difference Solvers on Structured Grids." *Computer Physics Communications*.

Jasak, Hrvoje, Aleksandar Jemcov, and Zeljko Tukovic. 2007. "OpenFOAM: A C++ Library for Complex Physics Simulations." *International Workshop on Coupled Methods in Numerical Dynamics*, 1–20.

Kok, Chiang Liang, Chee Kit Ho, Abbas Syihan Bin Muhammad Taufik, Yit Yan Koh, and Tee Hui Teo. 2024. "Development of a Sustainable Universal Python Code for Accurate 2D Heat Transfer Conduction Simulations in Educational Environment." *Applied Sciences* 14 (16): 7159.

LeVeque, Randall J. 2007. *Finite Difference Methods for Ordinary and Partial Differential Equations*. SIAM.

Liu, Xiao-Yu, Zhi Xie, Jian Yang, and Hong-Ji Meng. 2022. "Accelerating Phase-Change Heat Conduction Simulations on Gpus." *Case Studies in Thermal Engineering* 39: 102410.

Logg, Anders, Kent-Andre Mardal, and Garth N. Wells. 2012. *Automated Solution of Differential Equations by the Finite Element Method*. Springer.

Miotti, D., and others. 2021. "A Fully Meshless Approach to the Numerical Simulation of Heat Conduction Problems over Arbitrary 3D Geometries." *Energies* 14 (5): 1351. https://doi.org/10.3390/en14051351.

Patankar, Suhas V. 1980. *Numerical Heat Transfer and Fluid Flow*. Hemisphere Publishing Corporation.

Russell, MB, and SD Probert. 2004. "FDiff3: A Finite-Difference Solver for Facilitating Understanding of Heat Conduction and Numerical Analysis." *Applied Energy* 79 (4): 443–56.

Wei, Z., and others. 2014. "A Fast and Interactive Heat Conduction Simulator on Gpus." *Journal of Computational and Applied Mathematics*.

Weller, Henry G., G. Tabor, Hrvoje Jasak, and C. Fureby. 2007. "A Tensorial Approach to Computational Continuum Mechanics Using Object-Oriented Techniques." *Computers in Physics* 12 (6): 620–31.